

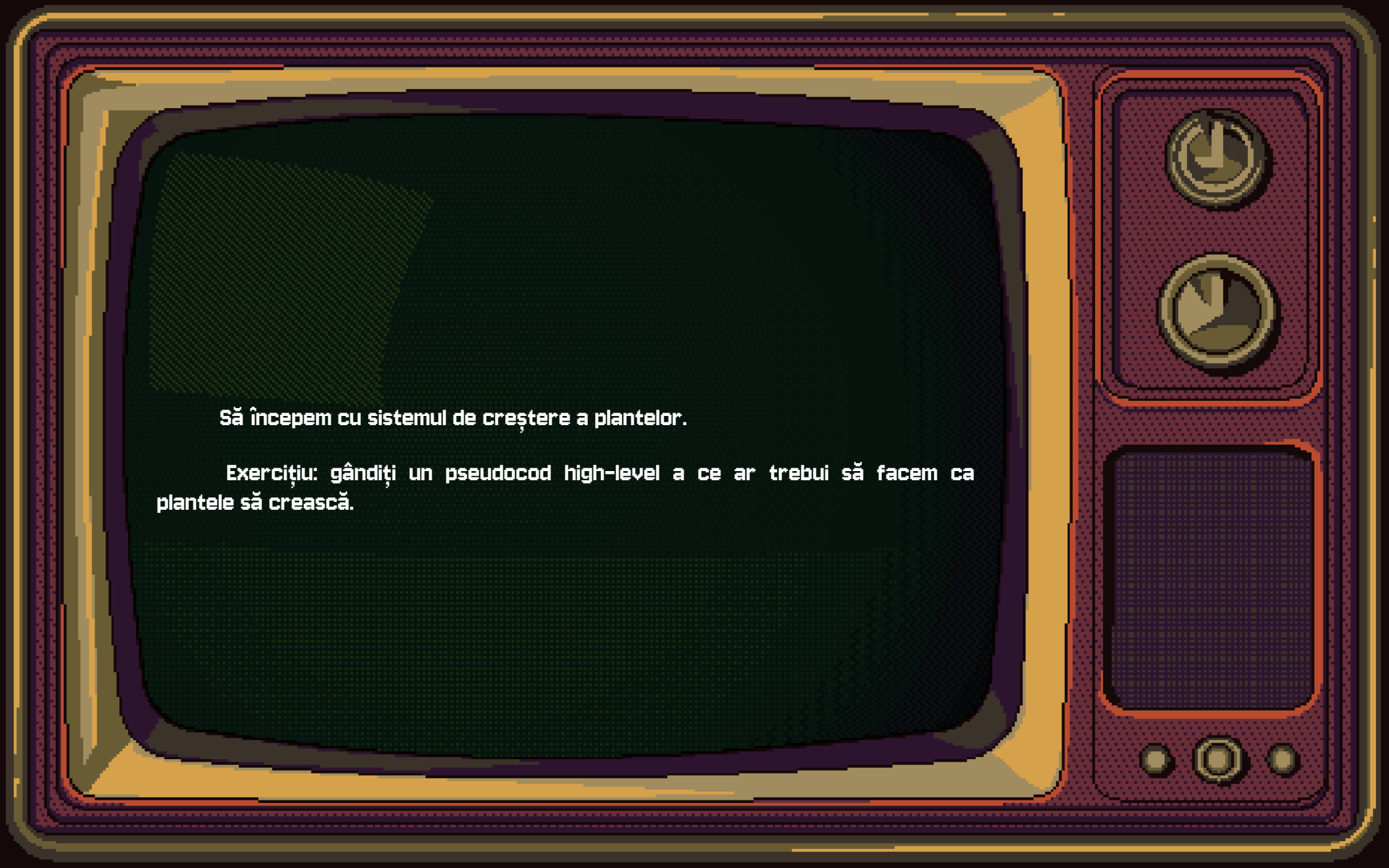
Prezentarea 5

Prof. Coordonator: Ș.I.dr.ing.Stelian-Nicolae NICOLA

Mentor: Pătrașcu Hary

The image shows a pixelated, retro-style interface. On the left is a large, dark, rounded rectangular area that serves as a main display or workspace. To the right of this area is a vertical control panel. At the top of the panel are two circular buttons, each with a small icon inside. Below these is a larger rectangular area with a grid pattern, possibly a menu or a list. At the bottom of the panel are three small circular buttons. The entire interface is framed by a thick, multi-layered border with a yellow and orange color scheme.

În prezentarea anterioară am reușit să implementăm funcționalitatea de plantare, dar mai există aspecte care trebuie implementate. În momentul actual plantele nu pot crește, jucătorul nu le poate colecta și nu poate selecta tipul de sămânță pe care vrea să o planteze.



Să începem cu sistemul de creștere a plantelor.

Exercițiu: gândiți un pseudocod high-level a ce ar trebui să facem ca plantele să crească.



1. Un sistem de timp

2. Când se schimbă ziua, să verific că tile-ul pe care se află planta este ud

Sistemul de timp(Timers)

Godot dispune de un nod numit Timer care este folosit pentru a implementa mecanici care necesită temporizare, de exemplu trecerea zilelor, cooldown-uri pentru abilități, spawnarea inamicilor la anumite intervale de timp, etc.



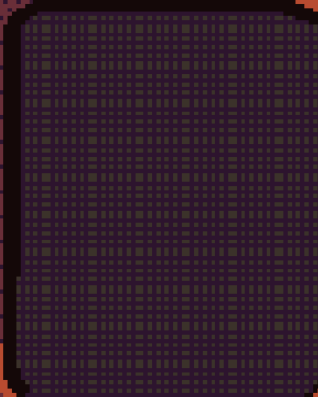
Exercițiu: Creați o scenă numită TimeManager cu nodul părinte Timer. Atașați un script acestei scene și adăugați scena în scena Level.

- Level
 - Environment
 - OceanLayer
 - GrassLayer
 - DirtLayer
 - WaterDirtLayer
 - Plants
 - Player
 - TimeManager

Filter Scripts

- level.gd
- time_manager.gd

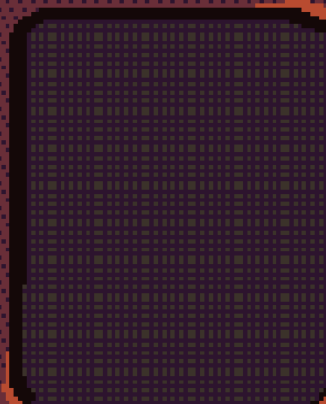
```
1 class_name TimeManager extends Timer
2
3 func _ready() -> void:
4     pass
5
```



Când selectați nodul timer puteți vizualiza toate proprietățile pe care le are. În tabul signals o să găsiți semnalul timeout care este foarte util în multe cazuri. În cazul nostru îl vom folosi pentru a declanșa anumite evenimente când se schimbă ziua. În funcție de wait time-ul pe care îl setăm, semnalul timeout se va declanșa după ce timer-ul a numărat până la waittime. De exemplu dacă setăm waittime-ul la 5 secunde, timeout se va declanșa după 5 secunde.

Exercițiu: setați proprietatea waittime la 5 și autostart pe true din editor.

Timerele pot fi create și din cod.





Acum că avem timer-ul, ar trebui să folosim semnalul de timeout.

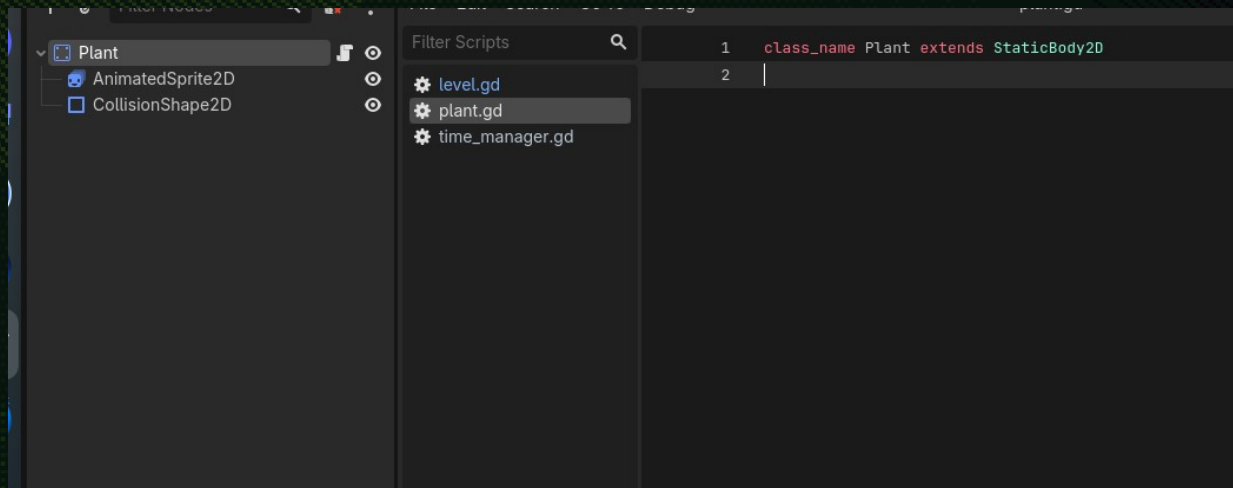
Exercițiu: conectați semnalul de timeout la scriptul level.



```
18  
→ 19 > func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void: ⌵  
58  
59  
→ 60 < func _on_time_manager_timeout() -> void:  
61 > | pass # Replace with function body.  
62
```



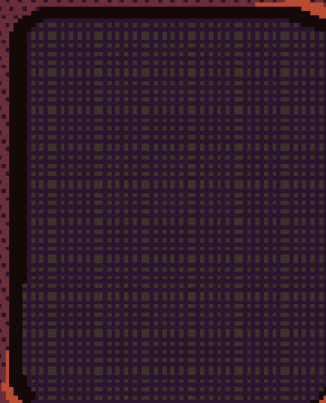
Acum că avem semnalul de timeout, ar trebui să verificăm plantele. În primul rând, să adăugăm un script la scena Plant.

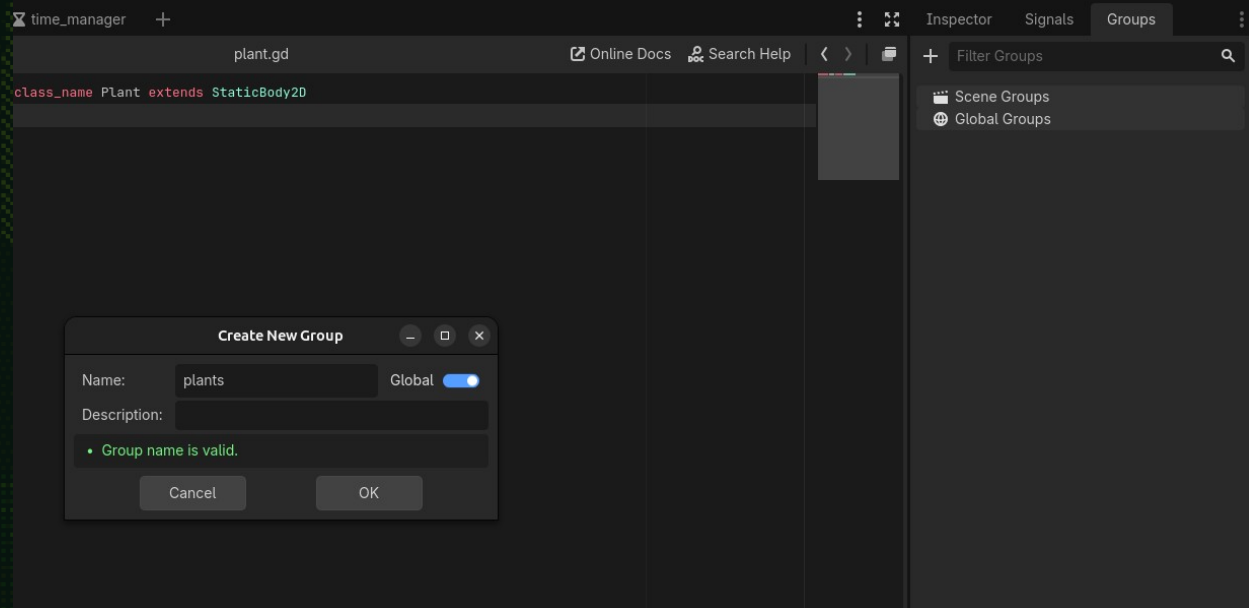


Grupuri de noduri(groups)

În Godot există o modalitate de a grupa noduri de un fel pentru a avea acces mai ușor la ele. În scena plant dacă vă uitați în dreapta o să vedeți taburile inspector, signals și groups.

Pentru a crea un grup mergeți în tabul groups și apăsați pe iconița '+' apoi dați un nume grupului, de exemplu plants și faceți grupul global pentru a putea fi accesat de către alte scripturi.







Acum din scriptul level când avem un timeout putem accesa toate plantele folosind acest grup și să apelăm o metodă de creștere a plantei de exemplu. Deoarece am făcut setările din editor, fiecare plantă pe care o instanțiem va fi adăugată automat în grupul de plante.

Exercițiu: accesați documentația Godot și încercați să găsiți o metodă care să vă dea toate nodurile dintr-un grup.



Metoda `get_nodes_in_group('nume_grup')` ne va da toate nodurile dintr-un grup. De asemenea există și metoda `call_group` care ne permite să apelăm o funcție pe toate nodurile dintr-un anumit grup.

Exercițiu: creați o metodă numită `grow_plant` în script-ul `plant.gd` și folosind metoda `call_group` apelați acea metodă din script-ul `level.gd` când aveți un timeout.



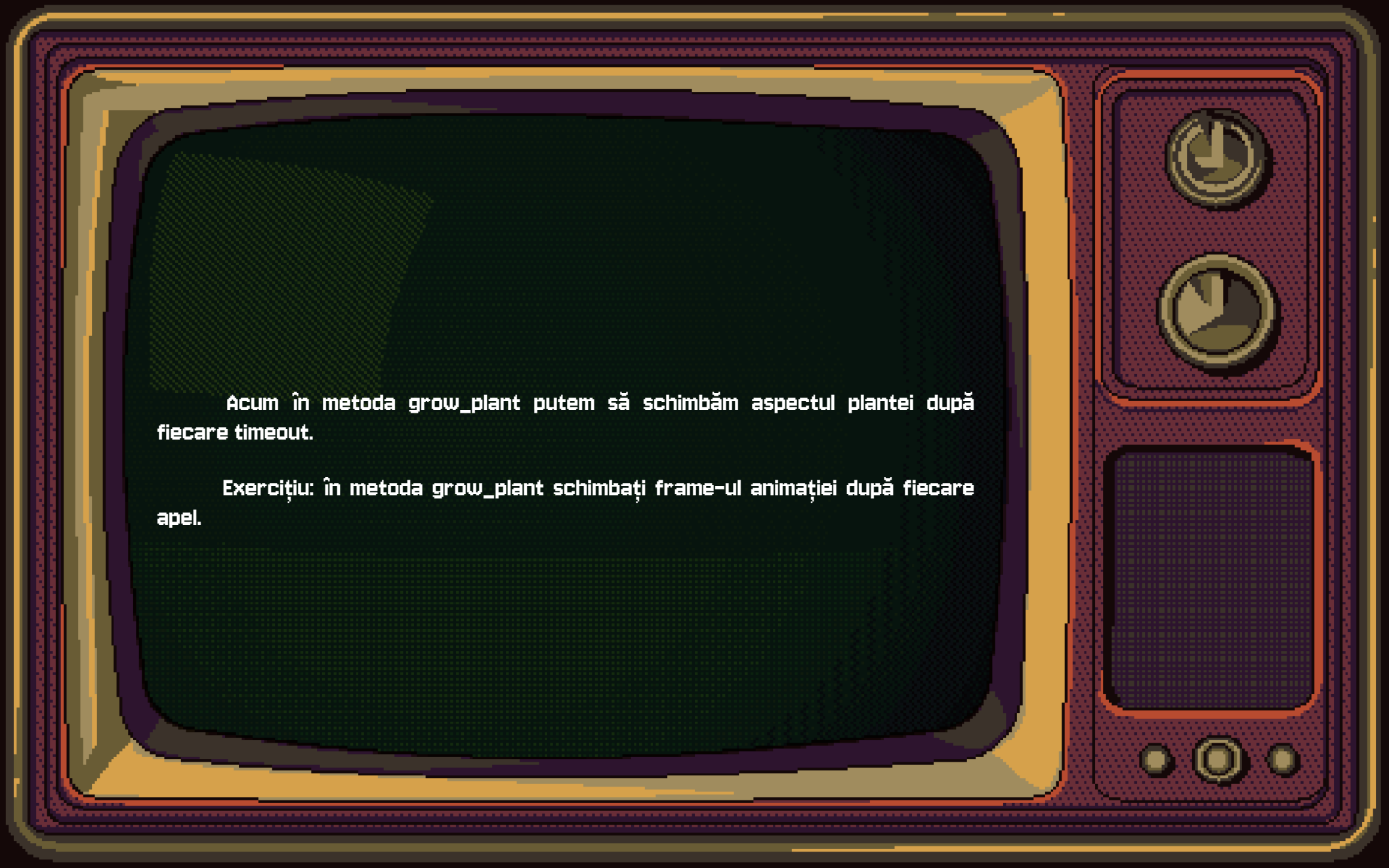
```
1  class_name Plant extends StaticBody2D
2
3
4  ▾ func grow_plant() -> void:
5    >| print("plant has grown")|
6    void print(...args: Array)
```

59

60 ▼ `func _on_time_manager_timeout() -> void:`

61 `> get_tree().call_group("plants", "grow_plant")`

62



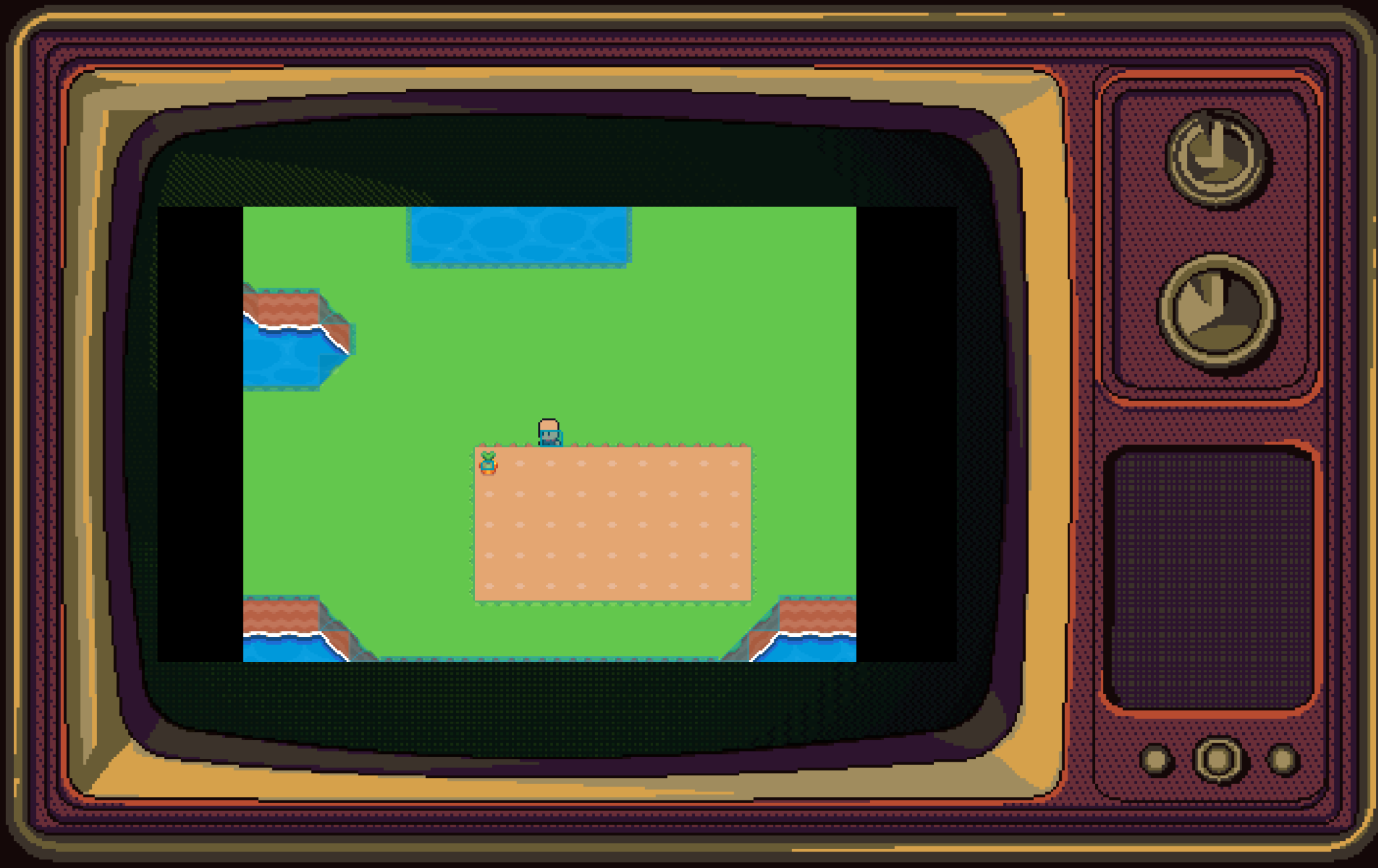
Acum în metoda `grow_plant` putem să schimbăm aspectul plantei după fiecare timeout.

Exercițiu: în metoda `grow_plant` schimbați frame-ul animației după fiecare apel.



În primul rând avem nevoie de o referință la nodul care are frame-urile anume `animatedsprite2d`. Apoi putem accesa proprietatea `frame` și să o incrementăm.

```
1  class_name Plant extends StaticBody2D
2
3  @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
4
5  func grow_plant() -> void:
6      >| print("plant has grown")
7      >| animated_sprite_2d.frame = animated_sprite_2d.frame + 1|
8
```





Mai trebuie să verificăm că planta nu a ajuns la ultimul frame pentru a
continua numărătoarea.

```
1  class_name Plant extends StaticBody2D
2
3  @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
4
5  func grow_plant() -> void:
6      >| # check the frame is less than the frame count and then increment the frame|
7      >| if animated_sprite_2d.frame < animated_sprite_2d.sprite_frames.get_frame_count("carrot"):
8          >|     animated_sprite_2d.frame = animated_sprite_2d.frame + 1
9          >|     print("plant has grown")
10
```





Următorul pas din pseudocod-ul nostru high-level este să verificăm că tile-ul pe care se află planta este udat pentru a continua procesul de creștere.

